

Video Summarization & Key Moments Detection Platform

Project Documentation

AI-powered video intelligence for faster learning, teaching, and content management.

1. Executive Summary & Project Objectives

ClipMind AI is an intelligent video intelligence platform engineered to ingest long-form media, perform speech-to-text transcription, generate abstractive text summaries, detect key topical transitions, and provide content analytics. The platform helps content creators, learners, educators, and administrators consume, teach, and manage video content with reduced scrubbing time.

2. Architecture & Technology Stack

Layer	Technology / Tool	Architectural Purpose
Presentation Tier	Next.js 14, React, TypeScript, Tailwind CSS, Lucide Icons	Responsive dashboards, reactive polling, role-based interfaces, and dark-mode styling.
API Gateway & Backend	FastAPI, Uvicorn ASGI, Python 3.11	Asynchronous REST endpoints, JWT authentication, RBAC authorization, and background workers.
Media Processing	FFmpeg	Binary stream demuxing, video thumbnail generation, and audio extraction into 16 kHz mono WAV streams.
Speech-to-Text (ASR)	OpenAI Whisper	Speech recognition with word/sentence timestamps and multi-accent support.
Summarization & NLP	Hugging Face Transformers (BART / T5)	Contextual token chunking, abstractive summaries, and sentiment classification.
Persistence Layer	PostgreSQL, SQLAlchemy ORM, MongoDB	Structured metadata, user roles, pipeline status, and full-text transcript storage.
Client Reporting	html2pdf.js, HTML5 Canvas	Client-side generation of downloadable A4 lecture briefs.
Containerization	Docker, Docker Compose	Multi-container orchestration with persistent volumes and isolated networking.

3. Milestone-by-Milestone Implementation Lifecycle

Milestone 1 (Weeks 1 & 2): Structural Setup, Media Ingestion & RBAC

- Full-Stack Initialization: Initialized the Next.js frontend and FastAPI backend with clear separation of concerns.
- Relational Storage & ORM: Configured SQLAlchemy with PostgreSQL for users and video metadata.
- Chunk-Based File Ingestion: Implemented disk-buffered ingestion in storage/raw_videos/ for .mp4, .mkv, .mov, and .avi files.
- FFmpeg Pipeline & Snapshotting: Extracted 720p preview thumbnails at the 2-second mark using an automated background task.
- Role-Based Access Control: Implemented Learner, Educator, Content Creator, and Administrator roles.

- Technical Challenge Resolved: Decoupled circular dependencies between database configuration and SQLAlchemy model imports by organizing initialization in main.py.

Milestone 2 (Weeks 3 & 4): Speech-to-Text & AI Summarization Pipelines

- Audio Extraction: Normalized audio into 16 kHz mono WAV files using FFmpeg.
- Whisper ASR Integration: Generated punctuated transcripts with word- and sentence-level timestamp alignment.
- BART Summarization Engine: Produced short 3–4 bullet executive summaries and detailed chapter breakdowns.
- Dual Storage Architecture: Used PostgreSQL for transactional metadata and MongoDB for searchable transcript documents.
- Asynchronous Execution: Used FastAPI BackgroundTasks to offload model inference and avoid UI timeouts.
- Technical Challenge Resolved: Addressed SQLAlchemy stale-read issues using db.expire_all() after background processing.

Milestone 3 (Weeks 5 & 6): Key Moments Detection, Analytics & Client Reporting

- Topic Segmentation: Used transcript phrase boundaries and Whisper audio chunks to identify topical shifts and clickable timeline markers.
- Content Intelligence: Calculated transcript word counts, processing durations, sentiment classifications, and BART compression efficiency.
- Entity & Keyword Tagging: Extracted high-density technical keywords and topics into an interactive tag cloud.
- Client-Side PDF Engine: Built browser-based A4 Highlight Reports using html2pdf.js.
- Technical Challenge Resolved: Fixed HTML5 canvas rasterization failures by replacing lab()/oklch() Tailwind colors with explicit hex values during DOM-to-canvas rendering.

Milestone 4 (Weeks 7 & 8): Model Benchmarking, Automated Testing & Containerization

- AI Model Benchmarking: Evaluated Whisper ASR and BART summarization performance.
- Whisper ASR: Reported 0.00% Word Error Rate (WER) with sub-second timestamp alignment precision.
- BART Summarizer: Reported 0.2143 F1 on ROUGE-1 and ROUGE-L, with approximately 90% content compression while retaining semantic context.
- Automated pytest Security Verification: Built and passed a four-test suite covering API health, upload authorization, write isolation, and cross-role access.
- Multi-Container Orchestration: Deployed PostgreSQL, FastAPI backend, and Next.js frontend using Docker Compose on a bridged network.

4. Empirical Evaluation & Quantitative Results

Word Error Rate (WER):

WER = (S + D + I) / N = (0 + 0 + 0) / 31 = 0.00%

Model / Pipeline	Metric Evaluated	Benchmark Target	Observed Result	Verdict
Whisper ASR	Word Error Rate (WER)	< 10.0%	0.00% WER (100% Match)	Passed
Whisper ASR	Timestamp Precision	< 1 s frame alignment	Sub-second accuracy	Passed
BART Summarizer	ROUGE-1 (Unigram F1)	Context retention	0.2143 F1	Passed
BART Summarizer	ROUGE-L (LCS F1)	Sentence structure	0.2143 F1	Passed
BART Summarizer	Information Compression	> 70% reduction	~90%–98.2% compression	Passed

Automated Security & Integration Test Suite (pytest)

- ✓ test_health_check_endpoint — PASSED
- ✓ test_rbac_learner_cannot_upload_directly — PASSED
- ✓ test_rbac_creator_can_upload_media — PASSED
- ✓ test_get_video_details_accessible_across_roles — PASSED
- Result: 4 passed, 0 failed in 2.00 seconds

5. Multi-Container Deployment Setup

The platform is managed through a single root docker-compose.yaml configuration. The deployment consists of three primary services: PostgreSQL, FastAPI backend, and Next.js frontend.

Service	Image / Build	Port	Purpose
clipmind-postgres	postgres:15-alpine	5432:5432	Persistent PostgreSQL database via pgdata volume.
backend	Custom image from ./backend/Dockerfile	8000:8000	FastAPI API, authentication, media processing, and AI inference.
frontend	Custom image from ./frontend/Dockerfile	3000:3000	Next.js user interface and dashboard.

Key Docker Configuration

```
services:
  clipmind-postgres:
    image: postgres:15-alpine
    container_name: clipmind-postgres
    ports:
      - "5432:5432"
    volumes:
      - pgdata:/var/lib/postgresql/data
    networks:
      - clipmind-network

  backend:
    build:
      context: ./backend
      dockerfile: Dockerfile
    container_name: clipmind-backend
    depends_on:
      - clipmind-postgres
    environment:
      - DATABASE_URL=postgresql://clipmind_user:clipmind_password@clipmind-postgres:5432/clipmind_db
      - WHISPER_MODEL_SIZE=base
      - BART_MAX_SUMMARY_LEN=140
    ports:
      - "8000:8000"
    networks:
      - clipmind-network

  frontend:
    build:
      context: ./frontend
      dockerfile: Dockerfile
    container_name: clipmind-frontend
    depends_on:
      - backend
    ports:
```

```
- "3000:3000"
networks:
  - clipmind-network

volumes:
  pgdata:

networks:
  clipmind-network:
    driver: bridge
```

6. Summary of Delivered Outcomes

- **Operational Video Intelligence Pipeline:** Integrated video upload, FFmpeg extraction, Whisper transcription, and BART summarization.
- **Verified RBAC Architecture:** Enforced role separation and write isolation across Creators, Learners, Educators, and Administrators.
- **Responsive Frontend & PDF Engine:** Delivered a Next.js 14 dashboard with live status polling and client-side PDF brief exports.
- **Containerized Deployment:** Packaged the platform into Docker containers orchestrated through Docker Compose for deployment readiness.

7. Role-Based Access Summary

Role	Access Level	Primary Capabilities
Learner	Read-only	Summaries, transcripts, timeline bookmarks
Educator	Manage	Lecture nodes, classroom share links, engagement tracking
Content Creator	Create / Edit	Media ingestion, transcript editing, key-moment curation
Administrator	Full Access	System analytics, resource monitoring, platform operations